DSMP LAB / TORONTO METROPOLITAN UNIVERSITY

DSMP Social Network Simulator User Guide

VERSION v2.6

Complete beginner guide for setup, dataset conversion, performance measurement, MLP testing, and team validation.

Release details

DSMP LAB

Software	DSMP Social Network Simulator
Branch / release	v2.6
Repository	github.com/aabhari/SocialNetworkSimulatorV3.1
Lab	DSMP Lab, Toronto Metropolitan University
Director	Dr. Abdolreza Abhari
Document version	Combined Final - August 2026

Testing priorities

- Convert new datasets into DSMP's common six-column follower-followee format.
- Test the updated MLP workflow, including custom settings and MLP Auto Search.

Contents

1. Purpose and Software Overview

DSMP is a Java multi-agent social-network simulation and recommender-system research tool with a Swing graphical interface. It can load social-network-style corpora, run recommendation or machine-learning algorithms, and measure accuracy and timing across one or more processing nodes.

The simulator standardizes input data into one shared representation. Twitter, retail, journal, and custom datasets must ultimately become the same six-column DSMP follower-followee text format before Performance Measurement.

Main workflow

Step	Action
1	Start DSMP
2	Use an existing DSMP dataset OR convert a new raw dataset
3	Switch to Performance Measurement
4	Load the DSMP text file
5	Set parameters and choose an algorithm
6	Get Users -> Initialize -> Run Simulation
7	Review recommendations, accuracy, timing, and logs

2. What Is New in v2.6

Area	v2.6 change	Tester focus
Dataset Import Lab	GUI import workflow for retail, journal, tweet, and custom sources.	Inspect, map, clean/balance, Build, then Use Output.
Class balancing	Optional imbalance diagnosis and balancing during import.	Compare unbalanced and balanced reports.
MLP Settings	Configurable layers, neurons, learning rate, max error, and max iterations.	Confirm custom values are applied.
Two MLP engines	Legacy Neuroph MLP plus experimental Sparse Federated MLP.	Test both on a small dataset before large runs.
MLP Auto Search	Automatic hyperparameter search over a loaded DSMP dataset.	Use Fast bounded first; verify Apply/Export.
Retail / sparse fixes	More stable sparse TF-IDF training/evaluation and accuracy reporting.	Prioritize retail MLP tests.
Startup / import robustness	Safer import handling and compile-before-run build flow.	Confirm clean setup on a fresh checkout.

Important: The current public v2.6 branch also contains the Dataset, Results, TestedDataset, TwitterGatherDataFollowers, Stored_NN, build.py, requirements.txt, pom.xml, and screenshot-instruction resources used by this guide.

3. Prerequisites

- Git
- Python 3.9 or 3.10
- JDK 8 or higher (JDK 8 preferred for compatibility)
- Apache Maven

All executable commands must be available on PATH.

```
git --version
python3.9 --version   # or python3.10 --version / py -3.9 --version
java -version
javac -version
mvn -version
```

4. Install and Start DSMP

4.1 Get the v2.6 code

```
git clone https://github.com/aabhari/SocialNetworkSimulatorV3.1.git
cd SocialNetworkSimulatorV3.1
git checkout v2.6
git branch --show-current   # expect: v2.6
```

4.2 First-time setup

```
Windows:      py -3.9 build.py setup
macOS/Linux:  python3.9 build.py setup
```

Setup creates the Python virtual environment, installs required Python packages and NLTK data, downloads Java dependencies with Maven, and compiles Java classes.

4.3 Run

```
Windows:      py -3.9 build.py run
macOS/Linux:  python3.9 build.py run
```

Keep the terminal open while DSMP runs. It is useful for progress and error messages.

The application opens in Twitter User Generation Simulation. Use the Performance Measurement button to switch to the recommender-system testing mode.

5. DSMP Data Format - Most Important Concept

Every Performance Measurement run expects a UTF-8, TAB-delimited .txt file with exactly six columns and no header row:

```
followee<TAB>postId<TAB>yyyy-MM-dd[ HH:mm[:ss]]<TAB>userId<TAB>userName<TAB>text
```

Column	Name	Meaning
1	followee / reference user	Class or recommendation target.
2	postId	Numeric tweet, invoice, paper, or generated record ID.
3	date	Prefer yyyy-MM-dd; time may be included.
4	userId	Numeric user, customer, or author ID.
5	userName	Follower/customer/author name - the entity that produced the text.
6	text	Tweet text, product description, paper title, or other textual content.

Examples

```
Twitter:  RyersonU<TAB>708526261514141696<TAB>2016-03-12 00:33:19<TAB>1<TAB>AHKru<TAB>release
force strong ...
Retail:   Sobeys<TAB>22153538971<TAB>2010-12-15 11:39<TAB>12865<TAB>FrankWarren<TAB>ANGEL
DECORATION STARS DRESS
Journal:  AmericanHeartJournal<TAB>4361535<TAB>1972-12-
31<TAB>67609887<TAB>OverallJamesC<TAB>intrauterine virus infections ...
```

In Twitter data, the reference account is the followee. In converted retail data, a store/category or mapped product category can be the followee and the customer becomes the user. In journal data, the journal/venue can be the followee and the author the user.

Ready-to-use repository datasets

File	Typical use
Dataset/Reduced_14k.txt	Small Twitter smoke test

Dataset/Retail.txt	Pre-converted retail test for SVM/MLP
Dataset/Reduced_57k.txt / Reduced_94k.txt	Larger Twitter tests
Dataset/Dataset-Journals-Authors-Titles-August6.txt	Journal recommendation experiments
TestedDataset/ and important-stuff/	Additional test splits / mappings

Important: Shared experiment datasets should be placed in Dataset/ so all testers use the same file. This is especially important for a requested large retail conversion such as a 100k-row test file.

6. Convert a New Dataset with Dataset Import Lab

Use this workflow whenever a raw dataset is not already in DSMP six-column format. This is the key missing workflow identified for the v2.6 guide.

6.1 Open Dataset Import Lab

1. Start the simulator.

2. On the User Generation screen, click Dataset Import Lab.

6.2 Choose dataset kind

Kind	Use when
Retail	UCI Online Retail / transaction exports such as CSV or Excel.
Journal	Author-title-venue tables.
Tweets	Twitter-like source files that still require cleaning/mapping.
Other	Delimited/workbook data where columns must be mapped manually.

6.3 Select source and Inspect

1. Browse to the raw file (.csv, .tsv, .xlsx, .xls, .txt, etc.).

2. Click Inspect.

3. Confirm delimiter, columns, row count, and preview are correct.

The importer can auto-detect common delimiters including comma, tab, semicolon, and pipe.

6.4 Choose a mapping profile

Profile	When to use
Similar users / entities	One shared collection label; useful for Doc2Vec / Similarity / K-Means.
Retail: smart product category	Recommended retail mapping for SVM and MLP tests.
Retail: legacy product family	Older keyword fallback for compatibility.
Retail: country baseline	Sanity check using country as the followee.
Retail: stock code experimental	Many stock-code classes; can stress classifiers.
Journal: authors by venue	Journal/venue = followee; title = text.
Tweets: users by reference account	Reference account = followee; tweet = text.
Other / custom DSMP mapping	Manually select followee, user, date, and text columns.

Important: For first retail MLP testing, use Retail: smart product category unless the experiment plan specifies another profile.

6.5 Cleaning and optional balancing

Review the source-column mappings and use cleaning options appropriate to the source. Retail imports may exclude cancelled transactions, missing customer IDs, invalid/non-positive quantity or price values, and unusable text.

Available balancing choices include:

- Off
- Diagnose only

- Conservative majority cap (recommended first balancing option)
- Strict user-level balance
- Hybrid cap + augmentation
- Weka SpreadSubsample / Resample

Important: For the first run, use Diagnose only. Inspect the class report before changing the data. If balancing is enabled later, record the mode, target, and random seed.

6.6 Build the output

1. Set Output directory - normally Dataset/ for shared testing.

2. Set an output name; do not overwrite the raw source.

3. Click Build.

4. Review the output preview and report.

5. Confirm each generated row follows the six-column format.

```
followee/referenceUser<TAB>numericPostId<TAB>yyyy-MM-dd<TAB>numericUserId<TAB>userName<TAB>text
```

6.7 Use the output

Click Use Output. The converted file is loaded into Performance Measurement so you can continue with Get Users -> Initialize -> Run Simulation.

6.8 Manual conversion alternative

1. Create six columns in this exact order: followee, postId, date, userId, userName, text.

2. Save as UTF-8 TAB-delimited text.

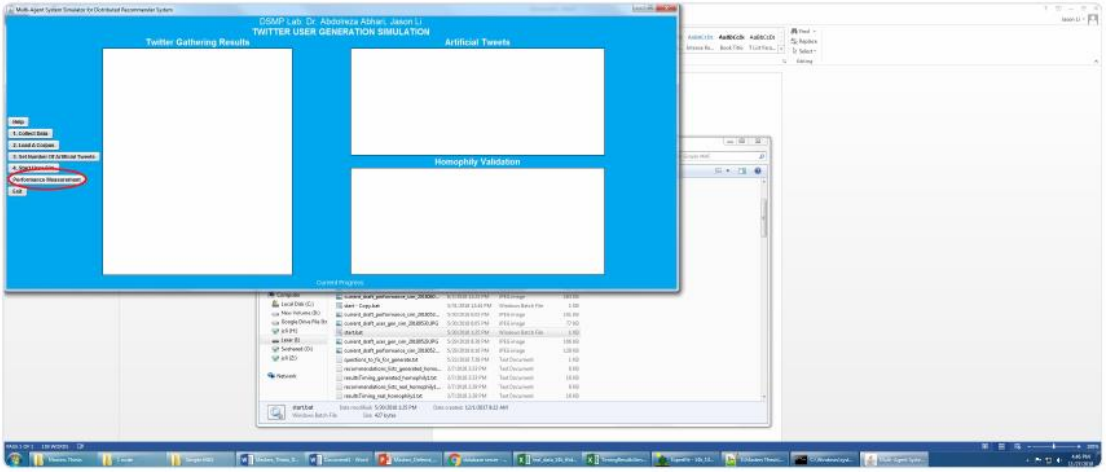
3. Remove the header row.

4. Place the file in Dataset/.

5. Load it using File -> Dataset From Text.

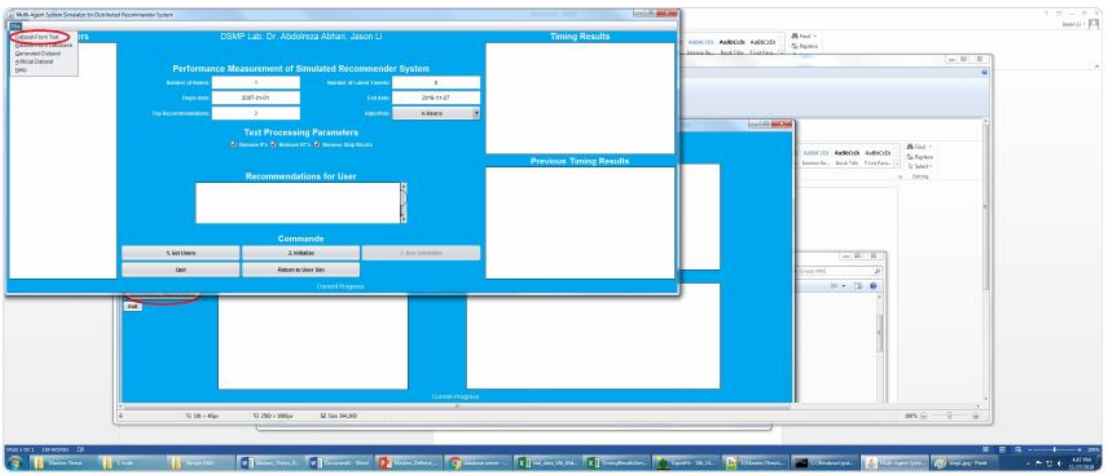
7. Performance Measurement - Beginner Walkthrough

The following figures are taken from the project's classic instruction set included in the supplied guide. The core button sequence remains useful in v2.6; Dataset Import Lab and the new MLP controls are additions.



1. On the User Generation screen, click **Performance Measurement**.

Figure 2 — Open a dataset from text



1. In Performance mode, open **File → Dataset From Text**.

Figure 3 — Choose a corpus file

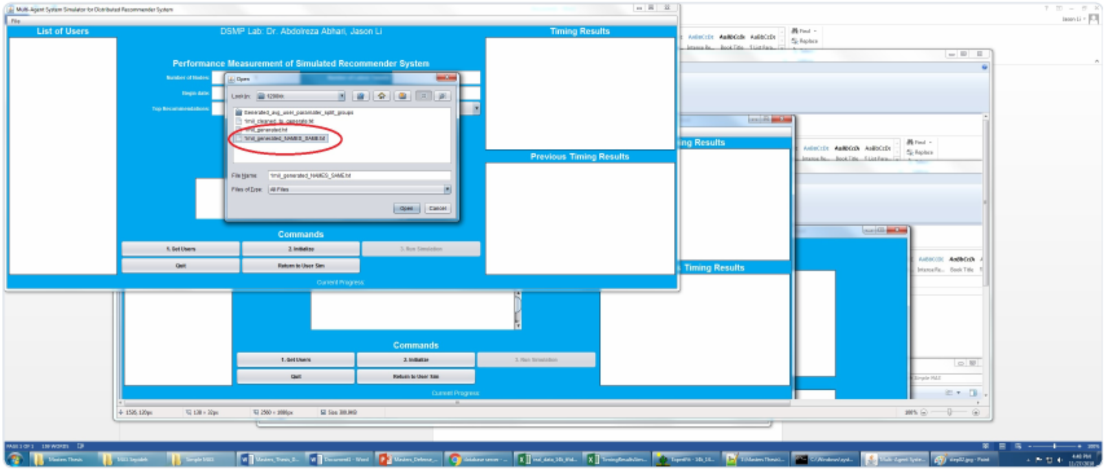
Figure 1-2. Switch to Performance Measurement and open File -> Dataset From Text. (Source guide page 9)

7.1 Load a dataset

1. Click Performance Measurement.

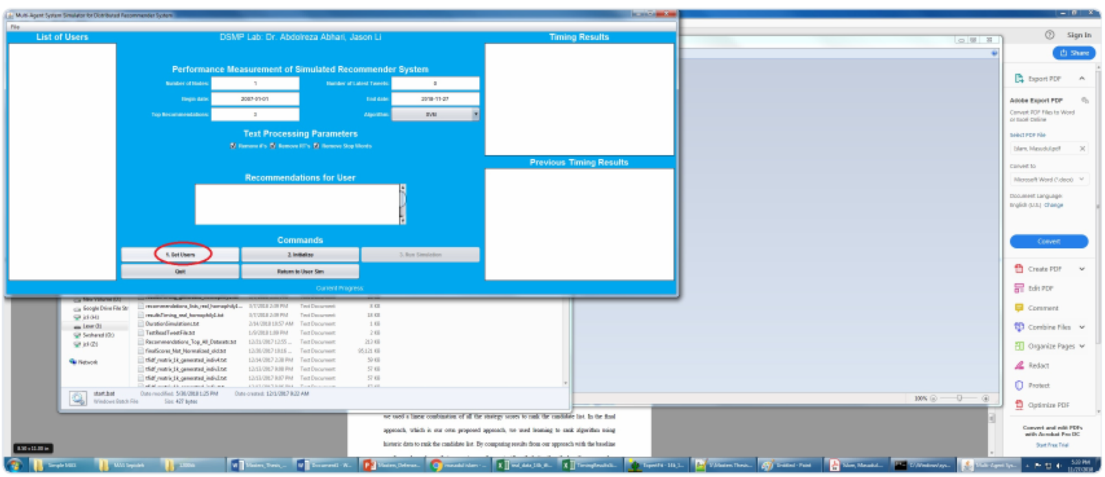
2. Open File -> Dataset From Text.

3. Choose Dataset/Reduced_14k.txt, Dataset/Retail.txt, or the new file generated in Section 6.



1. Select a ready DSMP file, for example:
2. `Dataset/Reduced_14k.txt` (quick Twitter check)
3. `Dataset/Retail.txt` (MLP retail test)
4. or your newly imported file from Section 6

Figure 4 — Set parameters, then Get Users



1. Set parameters, then click **1. Get Users**.

Suggested first-run values:

Figure 3-4. Choose a corpus, set parameters, and click 1. Get Users. (Source guide page 10)

7.2 Suggested first-run values

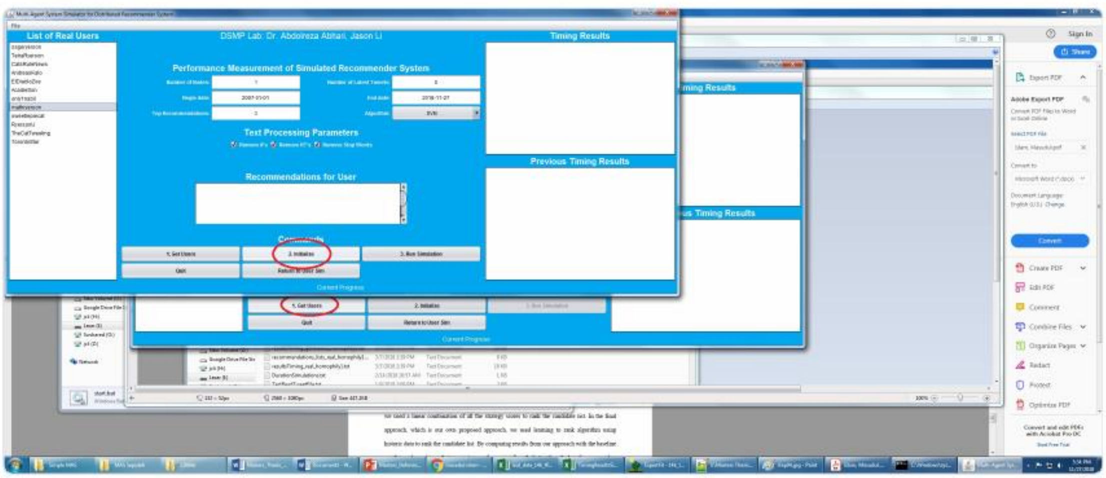
Parameter	Beginner value
Number of Nodes	1
Number of Latest Tweets	Leave default / use a reasonable limit
Begin date	Earlier than the oldest row; e.g. 2007-01-01 for older test corpora
End date	After the newest row

Top Recommendations	3
Algorithm	Start with SVM or MLP
Text processing	Optional Remove # / RT / Stop Words

Parameter	Beginner value
Number of Nodes	`1`
Number of Latest Tweets	leave default / reasonable limit
Begin date	`2007-01-01` (or earlier than your data)
End date	today / after your newest rows
Top Recommendations	`3`
Algorithm	start with `SVM` or `MLP`
Text processing	optional Remove # / RT / Stop Words

For MLP testing: choose **MLP** in the Algorithm dropdown. The **MLP Settings** and **MLP Auto Search** buttons become enabled.

Figure 5 — Initialize



1. After users appear in the list, click **2. Initialize**.

Figure 6 — Run Simulation

Figure 5. After users appear, click 2. Initialize. (Source guide page 11)

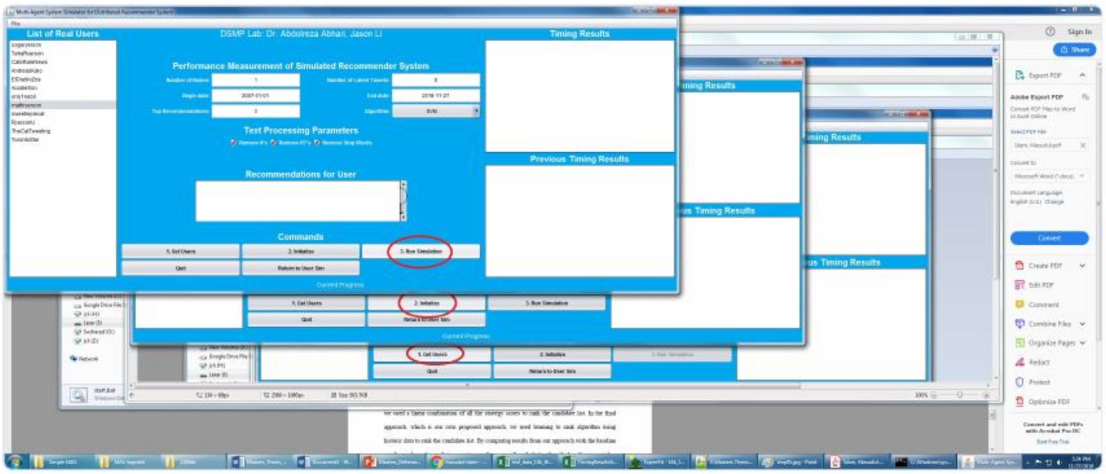
7.3 Initialize and run

1. Click 1. Get Users.

2. After users appear, click 2. Initialize.

3. Click 3. Run Simulation.

Important: If you change parameters, click Initialize again before the next run. If you load a different file, restart from File -> Dataset From Text.

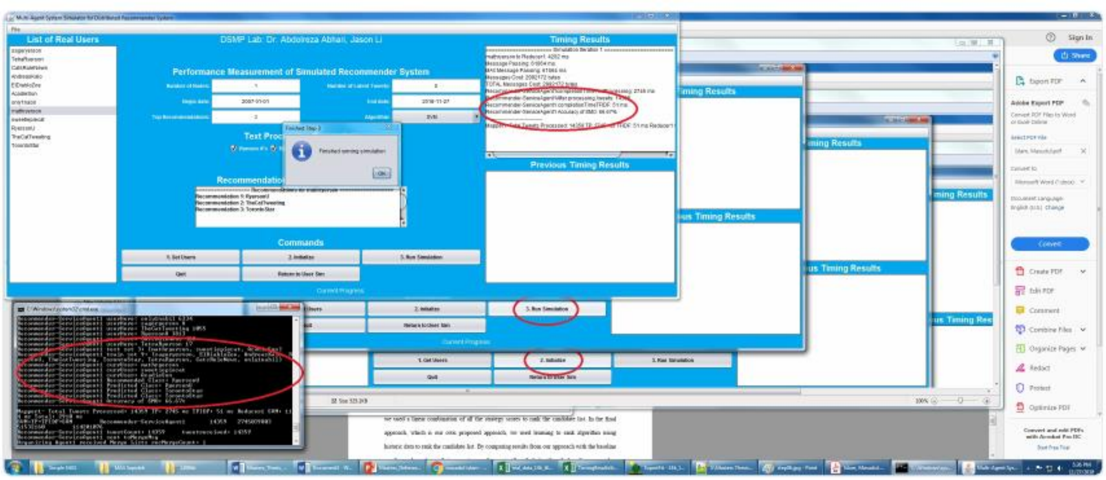


1. Click **3. Run Simulation**.

Important rules from the built-in Help:

- If you change parameters, click **Initialize** again before the next run.
- If you load a new file, restart from **File → Dataset From Text**.

Figure 7 — Read recommendations and timing



1. Inspect:

2. Recommendations for the selected user

3. Timing Results / Previous Timing Results

Figure 6-7. Run Simulation, then inspect recommendations, timing, and algorithm messages. (Source guide page 12)

7.4 Review results

- Recommendations for the selected user

- Timing Results and Previous Timing Results
- Accuracy / algorithm messages in result panels
- Terminal/log output for progress or errors
- Generated result files under Results/ or dataset-specific result folders

8. MLP in v2.6 - Complete Testing Guide

Important: MLP changed significantly in v2.6 and should receive extra testing.

8.1 Default Legacy Neuroph behavior

Setting	Default
Engine	Legacy Neuroph MLP
Hidden layers	1
Hidden neurons per layer	10
Learning rate	0.1
Max error	0.01
Max iterations	0 - no cap / unlimited

Leaving custom MLP settings disabled preserves the older Neuroph baseline for comparison with previous experiments.

8.2 MLP Settings dialog

1. Set Algorithm = MLP.

2. Click MLP Settings.

3. Check Use custom MLP settings.

4. Choose an engine and parameters.

5. Click Apply.

Engine A - Legacy Neuroph MLP

Field	Range / note
Hidden layers	1-8
Hidden neurons per layer	1-512
Learning rate	0.0001-1.0
Max error	>0 to 1
Max iterations	0 = uncapped; otherwise bounded, up to 100,000 in the v2.6 UI

Runtime warning: Deep uncapped Neuroph networks can run for many hours on retail/TF-IDF data. For testing, use a bounded max-iteration value or a bounded Auto Search profile.

Engine B - Sparse Federated MLP (experimental)

Field	Default / note
Hidden layers / neurons / learning rate	Configurable
Sparse epochs	80; range 1-500
FedAvg rounds	16; range 1-100
L2	0.0001 default in supplied guide
FedProx mu	0.0 default in supplied guide
Max error	Legacy-only

The supplied detailed guide notes that deeper sparse configurations use depth-stable leaky-ReLU training and cap effective learning rate at 0.03 for four or more hidden layers.

8.3 MLP Auto Search

1. Select Algorithm = MLP.

2. Click MLP Auto Search.

3. Choose/confirm the DSMP dataset.

4. Choose a search profile.

5. Review engine and candidate parameter values.

6. Set validation/test percentages and seeds if required.

7. Run the search and monitor progress/results.

8. Review the best configuration.

9. Apply the best settings back to MLP Settings or Export the report.

Profile	Intended use
Fast bounded search	First tester run / smoke search.
Balanced staged search	Stronger but still practical staged search.
Deep rescue search	Sparse-focused deeper rescue.
Legacy Neuroph bounded	Neuroph-only search with iteration caps.
Full research search	Exhaustive/overnight research search; approximately 28,800 planned runs in the supplied guide.
Custom	Manual grid.

Ranking uses validation accuracy first; test accuracy is for final audit. Bounded Neuroph candidates prevent an HPO search from running indefinitely.

Fast profile grid (documented default style)

- Layers: 1, 3, 5
- Neurons: 10, 25
- Learning rates: 0.1, 0.03, 0.01
- Legacy max iterations: 50
- Sparse epochs: 80, 160
- FedAvg rounds: 16

8.4 MLP tester checklist

1. Confirm the checked-out branch is v2.6.

2. Run Dataset/Retail.txt or Dataset/Reduced_14k.txt with Nodes = 1 and Algorithm = MLP.

3. First leave custom settings OFF to test the Legacy Neuroph default.

4. Repeat with bounded custom Neuroph settings, e.g. 1 layer / 10 neurons / LR 0.1 / max iterations 50.

5. Try a 3-layer Neuroph configuration with bounded iterations.

6. Try Sparse Federated MLP on Retail with defaults (80 epochs, 16 FedAvg rounds).

7. Run MLP Auto Search -> Fast bounded on the same dataset.

8. Apply the best settings and re-run Performance Measurement.

9. Optionally import raw UCI Online Retail using smart product category, save to Dataset/, and repeat.

10. Record dataset, engine, layers, neurons, learning rate, iterations/epochs, nodes, accuracy, wall-clock time, and completion status.

8.5 Long-running Neuroph note

The supplied testing notes document earlier 5-layer Neuroph runs on original unbalanced retail data taking approximately 47-71+ hours. The current Neuroph path is CPU-based, so a GPU machine does not automatically accelerate it. For routine testing, prefer bounded iterations, Sparse Federated MLP, or bounded Auto Search profiles.

9. Quick Start Recipes

Recipe	Steps
A - Fast Twitter smoke test	setup/run -> Performance Measurement -> Reduced_14k.txt -> Nodes 1 -> SVM/MLP -> Get Users -> Initialize -> Run
B - Retail MLP	Performance Measurement -> Retail.txt -> MLP -> optional MLP Settings/Auto Search -> Get Users -> Initialize -> Run
C - New raw UCI Retail	Import Lab -> Retail -> Inspect -> smart product category -> Diagnose only -> Build into Dataset/ -> Use Output -> MLP
D - Journal	Import Lab -> Journal -> authors by venue, or load the ready journal dataset -> Get Users -> Initialize -> Run

10. User Generation Simulation

Use this optional mode when generating artificial tweets from an existing corpus.

1. Collect Twitter data using the built-in workflow, or skip collection if a correctly formatted corpus already exists.

2. Load A Corpus.

3. Set Number Of Artificial Tweets (0 means use the processed corpus size).

4. Start User Sim / Start User Gen.

5. Evaluate the generated file later in Performance Measurement.

```
Collected: Dataset/TwitterObtained/<names>_<timestamp>/*_final.txt
Generated: Dataset/TwitterObtained/GENERATED/<ORIGINALFILENAME>_<timestamp>_GENERATED.txt
```

11. Algorithms Available

Algorithm	Notes
Similarity	Unsupervised similarity baseline
K-Means	Clustering recommender
SVM	Supervised baseline; useful retail/Twitter classifier check
MLP	v2.6 focus: Legacy Neuroph + Sparse Federated + Auto Search
Doc2Vec	Embedding-based path
Common-Neighbors	Graph-neighborhood recommendation
K-meansEuclidean	Euclidean K-Means variant

12. Multi-node / Parallel Experiments

1. Set Number of Nodes to 2, 4, or 8 as required.

2. Initialize again after changing node count.

3. Compare timing and accuracy against the 1-node baseline.

4. For Sparse Federated MLP, check Stored_NN/ for saved/averaged round models when produced.

If a deeper MLP gives only marginal accuracy improvement, use the smaller agreed architecture (for example a 3-layer configuration) for parallelization experiments to keep runtimes practical.

13. Recommended Validation Matrix

Test	Dataset	Configuration	Pass condition
Startup	Small existing DSMP file	Defaults	Application starts and completes a basic run.

Import - no balance	Small raw retail/custom	Balancing off/diagnose	Inspect -> Build -> Use Output works.
Import - balance	Same source	Chosen balancing mode	Report reflects changed class distribution.
Legacy MLP default	Small DSMP dataset	Custom off	Run completes and results are displayed.
Legacy MLP custom	Small DSMP dataset	Bounded iterations	Custom values are applied and run completes.
Sparse MLP	Retail/sparse dataset	80 epochs / 16 rounds baseline	Training completes and reports results.
Auto Search	Small DSMP dataset	Fast bounded	Search completes, best config shown, Apply works.
Large dataset	Approved shared large file	Known/bounded config	Progress, runtime, memory, and completion documented.
Multi-node	Same dataset/config	1 vs 2/4/8 nodes	Timing comparison recorded; re-initialization performed.

14. Troubleshooting

Problem	What to try
Wrong Python version	Use Python 3.9 or 3.10 with build.py.
java / javac / mvn not found	Install JDK/Maven and add commands to PATH.
Setup fails	Run build.py info, then setup -v.
Dirty/stale build	Run build.py clean, then build.py setup.
Get Users finds nobody	Check six-column TAB format and Begin/End dates.
Import looks wrong	Re-Inspect; check delimiter, dataset kind, mappings, and profile.
Output has too few rows	Review cancellation, missing-ID, quantity/price, and text filters.
MLP buttons disabled	Select Algorithm = MLP first.
Neuroph never finishes	Bound max iterations, use bounded Auto Search, or switch to Sparse Federated.
Auto Search is too large	Use Fast bounded; avoid Full research for quick testing.
UI appears frozen	Check terminal/log progress before stopping.
New dataset not visible to others	Save the converted DSMP file in the shared Dataset/ folder.

15. Development / Repository Rules

For simulator code, the lab's stated process is incremental and sprint-like. Avoid conflicting pushes to the shared active software version. Code changes should be unit-tested, documented in a change list, and reviewed before merge. One designated developer should coordinate software pushes to the shared repository.

- Do not push unreviewed simulator code directly to the shared active version.
- Keep experimental code local or on an agreed branch until unit-tested.
- Prepare a concise change list before merge/review.
- Documentation and other non-functional work may be developed on a separate branch.
- For every bug report include branch/version, dataset, settings, exact reproduction steps, expected result, actual result, and relevant logs.

16. One-page Cheat Sheet

1. Clone repository and checkout v2.6.

2. Run Python 3.9/3.10 build.py setup.

3. Run build.py run.

4. If using raw data: Dataset Import Lab -> Inspect -> Map -> Clean/Balance -> Build into Dataset/ -> Use Output.

5. Performance Measurement -> File -> Dataset From Text.

6. Set parameters -> Get Users -> Initialize -> Run Simulation.

7. For MLP: start with default Legacy Neuroph; then test custom bounded settings and Sparse Federated.

8. For Auto Search: start with Fast bounded.

9. Re-Initialize after parameter changes.

10. Record accuracy, timing, architecture, engine, dataset conversion settings, and completion status.

```
Canonical DSMP line:
followee<TAB>postId<TAB>yyyy-MM-dd<TAB>userId<TAB>userName<TAB>text
```

Appendix A - Figure Index

Figure	Source	Purpose
1-2	Supplied guide page 9	Switch to Performance Measurement; File -> Dataset From Text
3-4	Supplied guide page 10	Select corpus; set parameters; Get Users
5	Supplied guide page 11	Initialize
6-7	Supplied guide page 12	Run Simulation; read recommendations/timing

Appendix B - Sources and Verification

This combined final version consolidates the two supplied user guides and cross-checks installation/repository details against the public v2.6 GitHub branch.

- Supplied file: DSMP_v2.6_User_Guide_Final(1).docx
- Supplied file: DSMP_User_Guide_v2.6.pdf
- Public v2.6 branch README and repository tree
- Project Dataset/ and screenshot-instruction resources
- DatasetImportLabDialog / UciRetailDatasetImporter references documented in the supplied guides
- MlpSettingsDialog / MlpHyperparameterSearchDialog references documented in the supplied guides
- In-app ControllerAgentGui Help summarized in the supplied detailed guide

Repository:

`https://github.com/aabhari/SocialNetworkSimulatorV3.1`

v2.6 branch:

`https://github.com/aabhari/SocialNetworkSimulatorV3.1/tree/v2.6`

End of DSMP v2.6 Combined Final User Guide.